Intro to Machine Learning

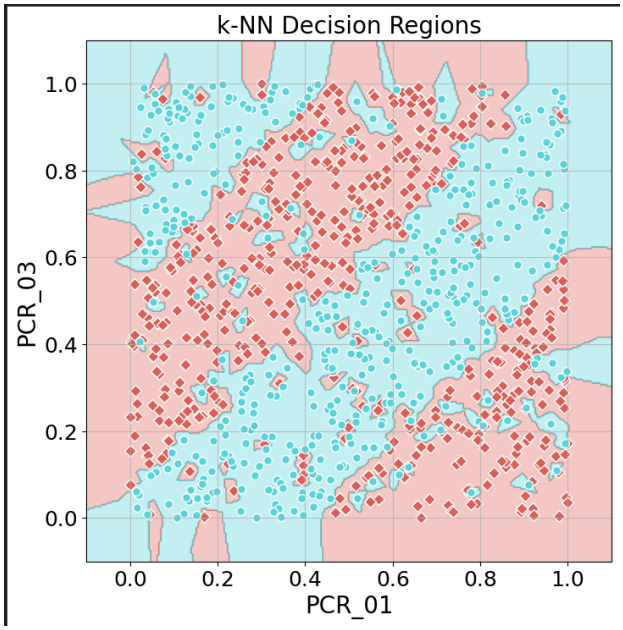
Major HW2

Ameer Muslih – 212209241

Mohamad Safadi - 211519467

Part 1

Q1

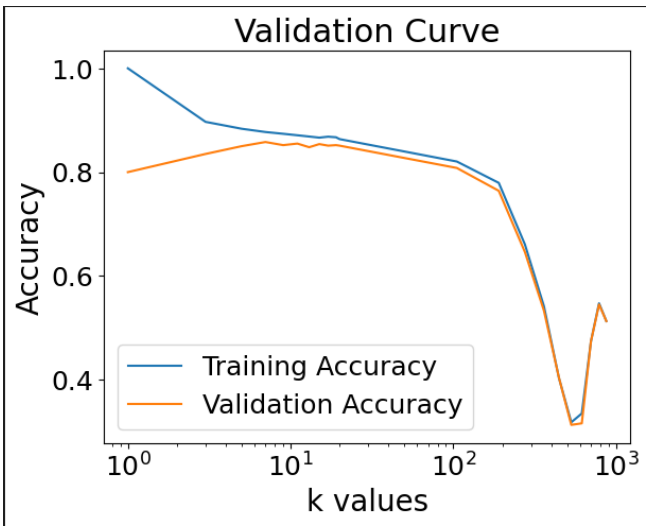


Q2

Best k value: 7
validation accuracy: 0.858
training accuracy: 0.8772857142857142

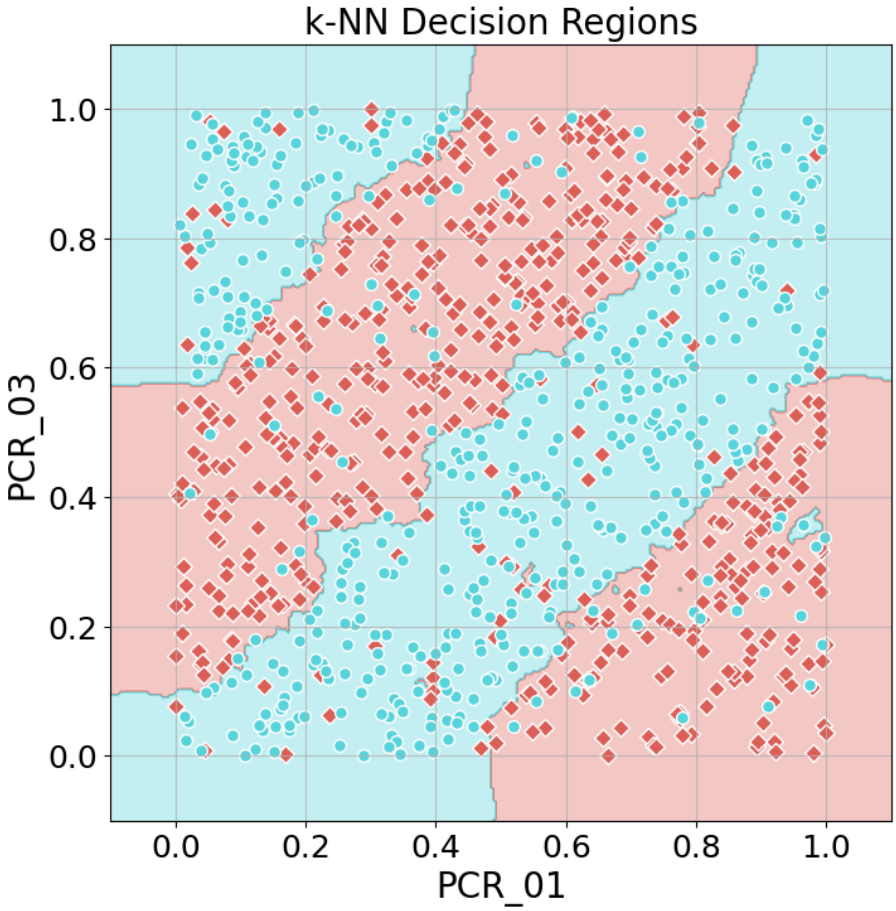
We can see that for K=1 we have overfitting, where we get a training accuracy of 1.0, that's because the model become too specialized for the training data as each prediction for a new data point is based solely on the label of the single nearest neighbour in the training set.

And we also notice the for k>300 we have a case of underfitting where the training Accuracy goes under 0.6, this happens because our K is too high in regard to the data set and the model becomes too generalized and stops capturing the patterns in our data.



Q3

Test Accuracy: 0.848



Q4

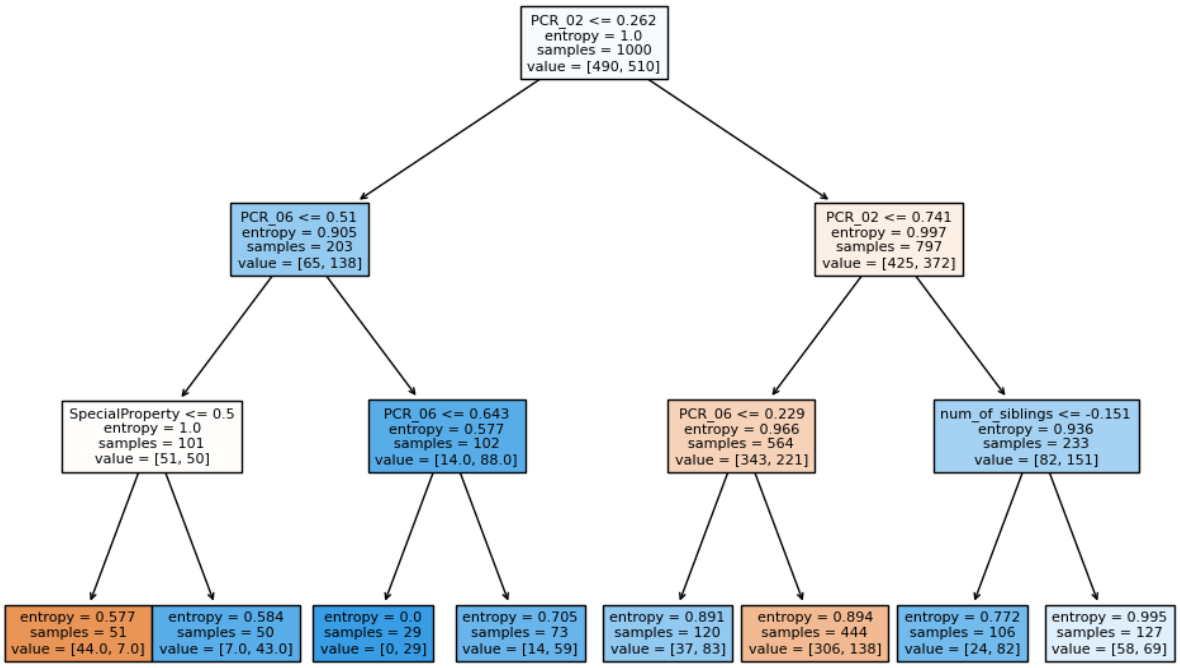
The classification boundaries exhibit greater smoothness with k=7, indicating reduced reliance on individual data points. This results in improved overall accuracy, as the model is not overly fixated on specific training instances but instead aligns better with the general distribution of the problem.

Notably, the test accuracy for k=7 is 0.848, outperforming the accuracy of 0.79 observed when k=1.

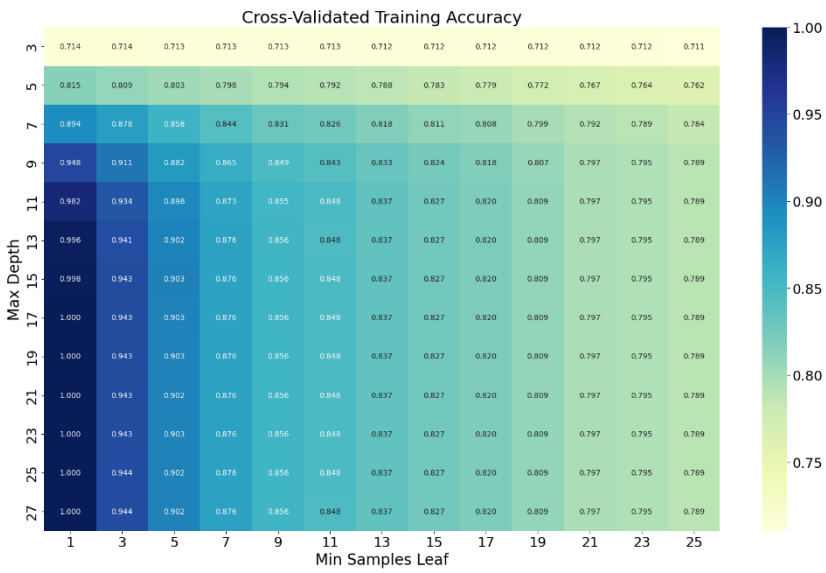
Part 2

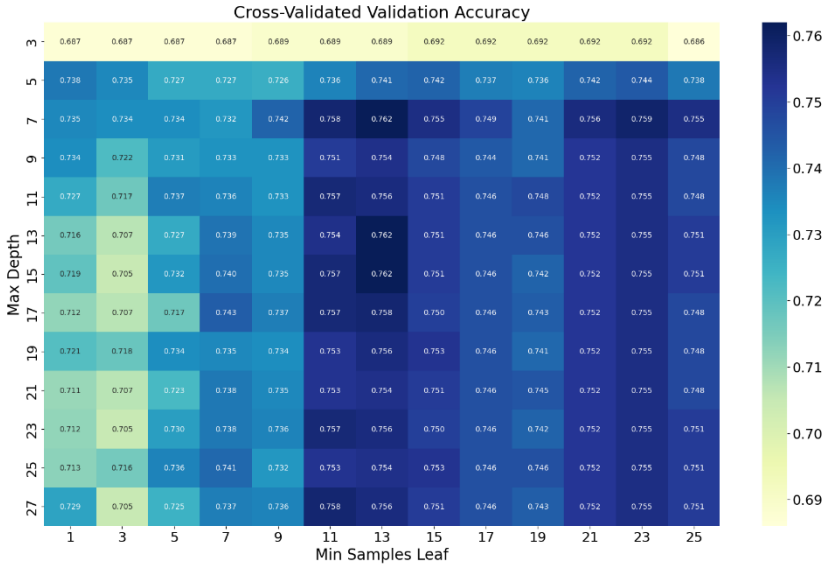
Q5

Training Accuracy: 0.715



Q6





Optimal hyperparameter combination

max_depth=7, min_sample_leaf=13 - val_accuracy: 0.762

Underfitting

max_depth=3, min_sample_leaf=15 - training_accuracy: 0.712

These hyperparameters impose strict constraints on the model, as were lowering the max_depth and taking the min_sample_leaf higher. The resulting lack of flexibility prevented the model from adequately fitting the training data.

Overfitting

max_depth=17, min_sample_leaf=1 - training_accuracy: 1.0

In this case the hyperparameters are too flexible in regard to our data, which leads the model to adapt too closely to the training data and even memorize noise and outliers rather than learning the generalizable patterns, thus we got overfitting.

Q7

In our grid search, we evaluated 8 different values for each hyperparameter, resulting in a total of 64 combinations to fine-tune the Decision Trees model. If we were to introduce a third hyperparameter with 8 different values for example, the number of combinations would increase to 512.

The impact of searching over additional hyperparameters is exponential, as each new hyperparameter multiplies the number of combinations by the number of values it can take. This exponential growth in combinations can lead to a significantly longer tuning process. Consequently, it becomes crucial to strike a balance between the number of hyperparameters explored and the computational resources available.

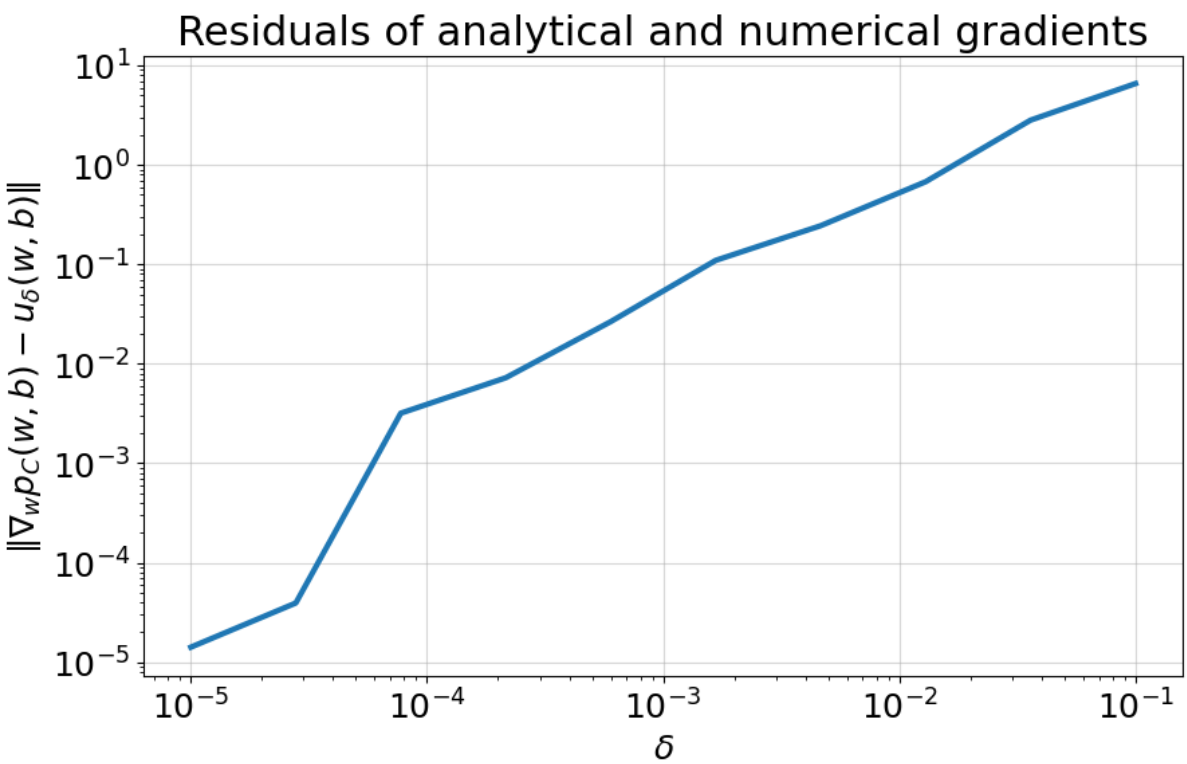
Q8

Test Accuracy: 0.7

Part 3

Q9

We can see that increasing the step size (delta) results in a larger discrepancy between the two gradients. This aligns with the expected behaviour of numerical differentiation, where a smaller delta provides a more accurate approximation to the analytical computation.
with a small enough delta we can see that both computations are fairly equal.



Q10

We can notice that we have a very large C and a considerably small lr.
The large C leads our model to have a very low tolerance to violations, we learned in the tutorials that for such a large hyperparameter C we get a HardSVM like model, but as our data isn't linearly separable we don't get a good accuracy convergence here.

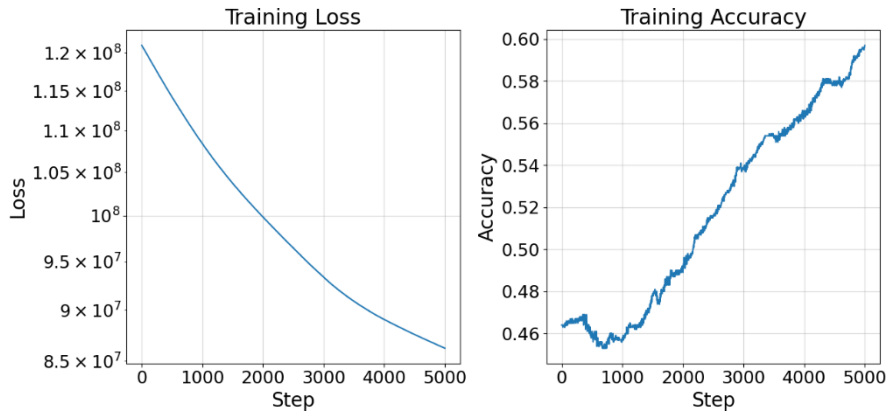
Regarding the loss behavior, we can see a rapid convergence with the loss function which is fairly stable and continuous because of the low learning rate, on the first couple of iterations the model is able to minimize the loss by a fair bit and that is because we start with a random w and we can get to a better fitting quite fast, and even a small change in our hinge loss sum equates to a large total loss reduction because of the large C, afterwards the change in loss is very minor because we have a small learning rate and change in the w happen very slowly.

Regarding the training accuracy, we can see that within the given iterations the training accuracy fluctuates almost around the same value and that is because of the small learning rate, as it would take way more iterations for our model to get any good training accuracy, although as stated before with the given hyperparameter C and our data we won't reach a good accuracy because of the large penalty on violations, this phenomenon arises from the interplay between the large C value, enforcing strict margin violations, and the small learning rate, which slows down parameter updates. Consequently, while the model strives to minimize the loss further, the incremental adjustments become increasingly marginal.

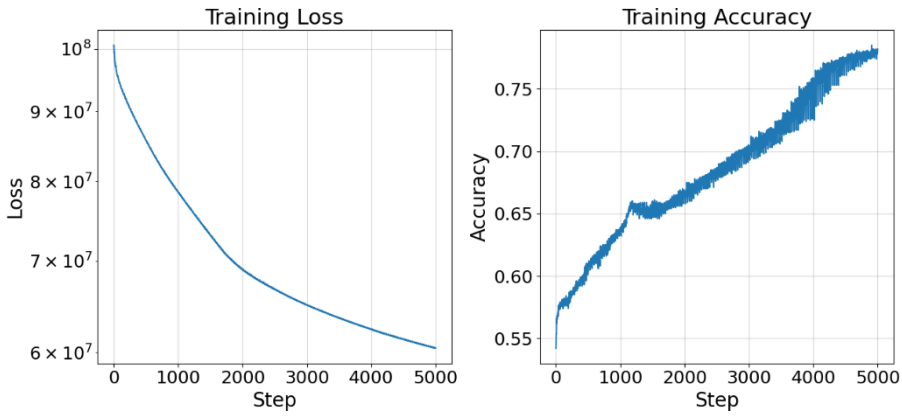
We can see that at the start when the loss was going down our training accuracy went up as expected, but then we hit somewhat of a plateau state where a drastic change in training accuracy would take a-lot of iterations.

Q11

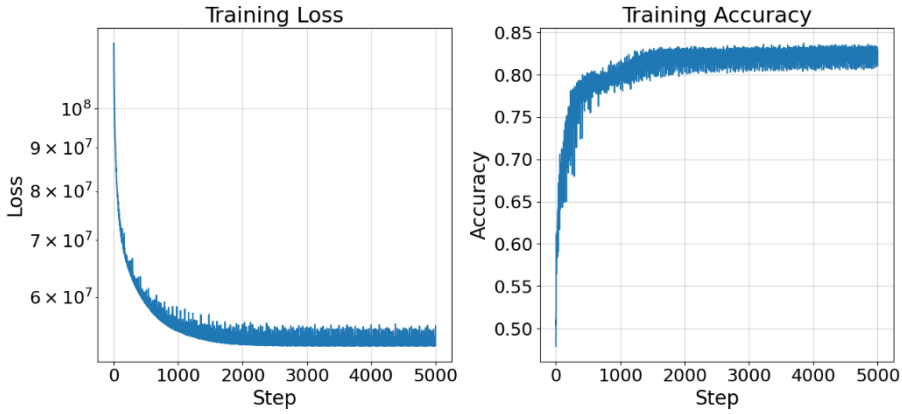
- lr= 1e-9



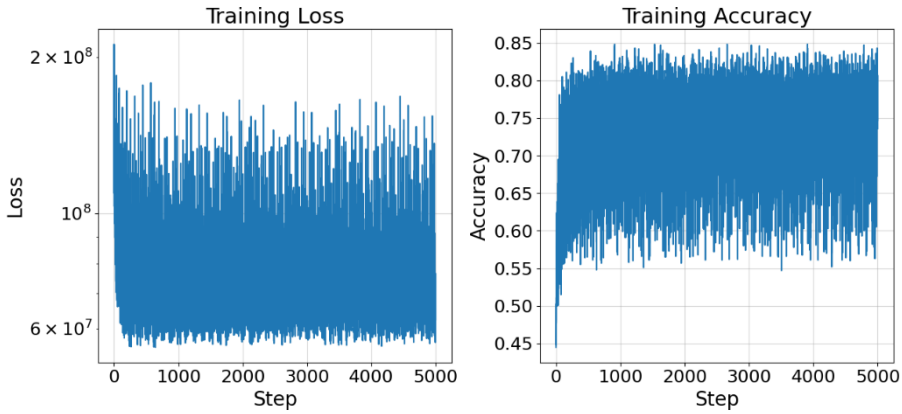
- lr= 1e-8



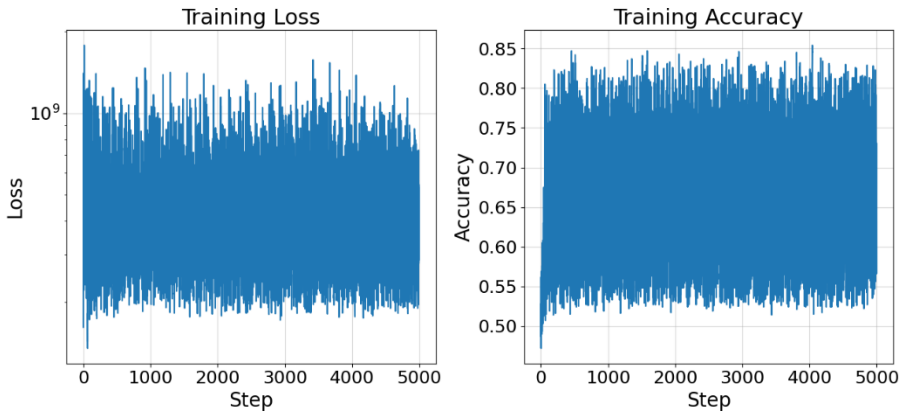
- lr= 1e-7



- lr= 1e-6



- lr= 1e-5

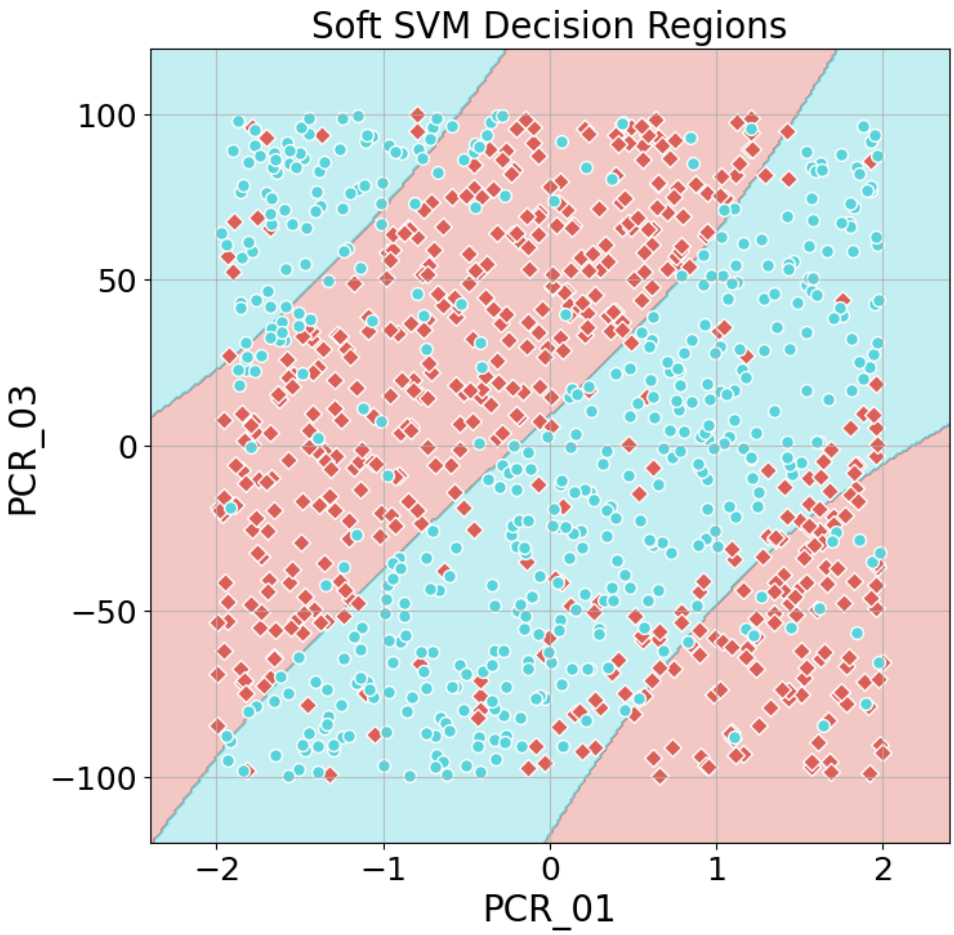


I would choose a learning rate of 1e-7, because if we compare the Training Accuracy plots we can see that for our selected value we get the fastest convergence rate and the highest training accuracy within the 5000 iterations, also looking at the training loss plots we can see that our loss quickly converges and stabilizes on a low value compared to other plots

Q12

Training Accuracy: 0.811

Test Accuracy: 0.832



Part 4

Q13

a.

$$\lim_{\gamma\to 0} sign\left(\sum_{i\in[m],\alpha_i>0}\alpha_i y_i e^{-\gamma\|x-x_i\|_2^2}\right)\overset{1}{=} sign\left(\lim_{\gamma\to 0}\sum_{i\in[m],\alpha_i>0}\alpha_i y_i e^{-\gamma\|x-x_i\|_2^2}\right)\overset{2}{=}$$

$$= sign\left(\sum_{i\in[m],\alpha_i>0}\alpha_i y_i\left(\lim_{\gamma\to 0} e^{-\gamma\|x-x_i\|_2^2}\right)\right)\overset{3}{=}$$

$$= sign\left(\sum_{i\in[m],\alpha_i>0}\alpha_i y_i * 1\right)\overset{4}{=} sign\left(\sum_{i\in[m],\boldsymbol{s.t.y_i=-1},\alpha_i>0}\alpha_i y_i + \sum_{i\in[m],\boldsymbol{s.t.y_i=1},\alpha_i>0}\alpha_i y_i\right)\overset{5}{=}$$

$$= sign\left(\sum_{i\in[m],\boldsymbol{s.t.y_i=1},\alpha_i>0}\alpha_i - \sum_{i\in[m],\boldsymbol{s.t.y_i=-1},\alpha_i>0}\alpha_i\right)\overset{6}{=} argmax_{y\in\{-1,1\}}\sum_{\{i|y_i=y\}}\alpha_i$$

Explanations for our steps:

1. As stated in assumption 1, we can use lim(sign)=sign(lim).
2. As i, α_i and y_i aren't related to γ we can take the limit into the sum and use only on our exponent.
3. Based on assumption 2, the norms of all feature vectors x are bounded, i.e. $\|x\|_2 < \infty$, thus we get from the triangle inequality that $\forall i$: $\|x - x_i\|_2 \leq \|x\|_2 + \|x_i\|_2 < \infty$.
   so when we take $\gamma \to 0$ we basically get $\lim_{\gamma\to 0} e^{-\gamma * Const} = 1$.
4. Here we split our sum into two separate sums where the first sums α_i for i's where $y_i = -1$ and the second is for $y_i = 1$, which together add up to the original sum.
5. Substituted the value of y_i for each sum.
6. Based on how the sign function works the previous equation returned 1 iff $\sum_{i\in[m],\boldsymbol{s.t.y_i=1},\alpha_i>0}\alpha_i > \sum_{i\in[m],\boldsymbol{s.t.y_i=-1},\alpha_i>0}\alpha_i$, meaning when the sum of α_i for i's where $y_i = 1$ is greater than the sum of α_i for i's where $y_i = -1$ we get 1 and vice versa, this is the exact behaviour we get from our argmax function, as we take the sum of α_i's for i s.t. $y_i = y$, and our argmax takes the maximum out of the two options $y \in \{-1,1\}$.

b. Based on our proof for an SVM model with an RBF kernel when $\gamma \to 0$, we get that for the case where $\alpha_i = 1, \forall i \in [m], \forall x \in \mathbb{R}^d$:

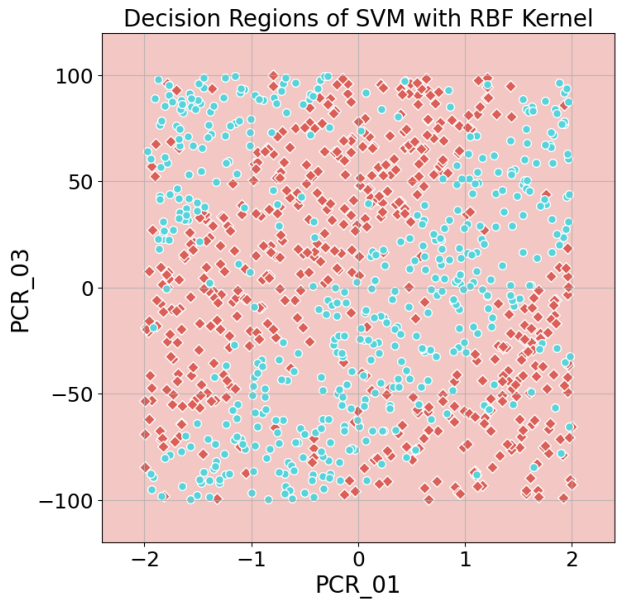
$$h(x) = argmax_{y\in\{-1,1\}}\sum_{\{i|y_i=y\}} 1$$

Meaning that for every new point x our classification will be the majority label for all train datapoints.

This model is similar to KNN where we take K to be equal to the number of training datapoints we have, to be more general we can say that our model is like KNN with $K \to \infty$.
also from a quick google search it seems like there's a model called majority class that behaves like this.

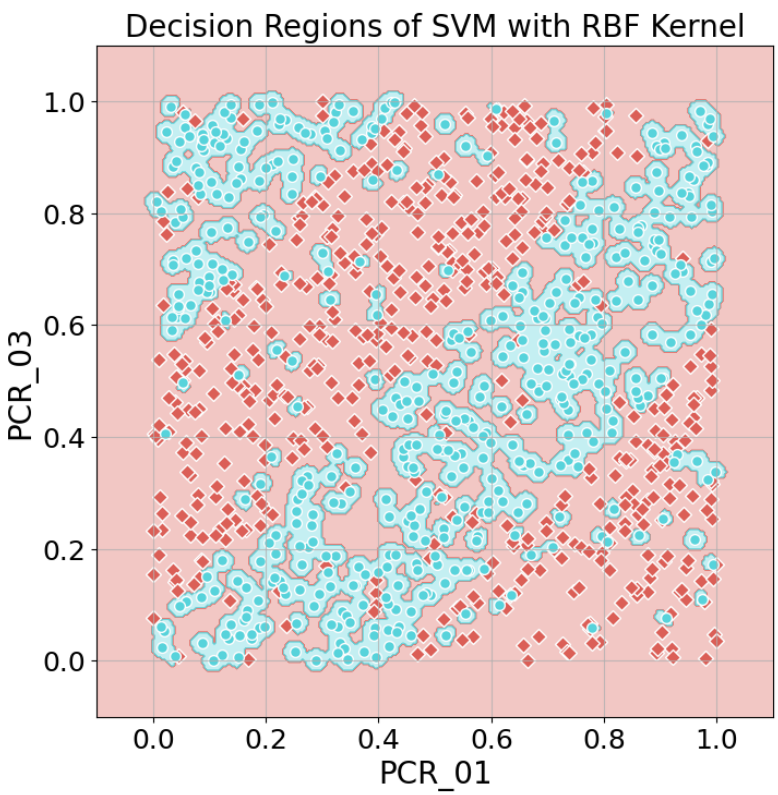
Q14

We can see that all of our decision region is classified the same (classified as {-1}), because if we take a look at our Y_train we can see that we have 513 datapoint with a {-1} label and 487 datapoints with label {1}, meaning the majority label is -1 thus as explained in the previous question we'll get the following decision region:

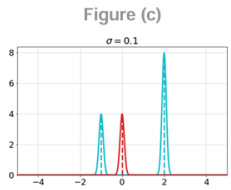


Q15

We can notice that for $\gamma = 5000$ our decision region looks different to the 1-NN decision region.



As we can see in this case we get {1} classification only for a very close proximity of each blue datapoint, we can think of it as described in this figure.



The reason our decision region looks as follows, is that when we have a new point x such that for a given x_i, $\|x - x_i\|_2^2$ isn't very small we get that $e^{-\gamma\|x-x_i\|_2^2}$ is nearly zero (similarity is zero), meaning that the training data point x_i doesn't affect the classification, as opposed to 1-NN where we look for the nearest neighbor without regard to how far that neighbor is.

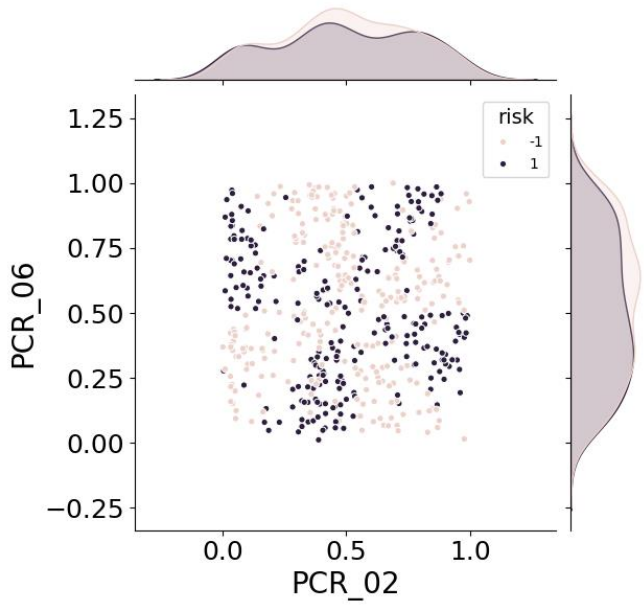
And for the case where our new point x has a very close training datapoint x_i such that $\|x - x_i\|_2^2$ is very small, the classification for x will be y_i.

Part 4

Q16

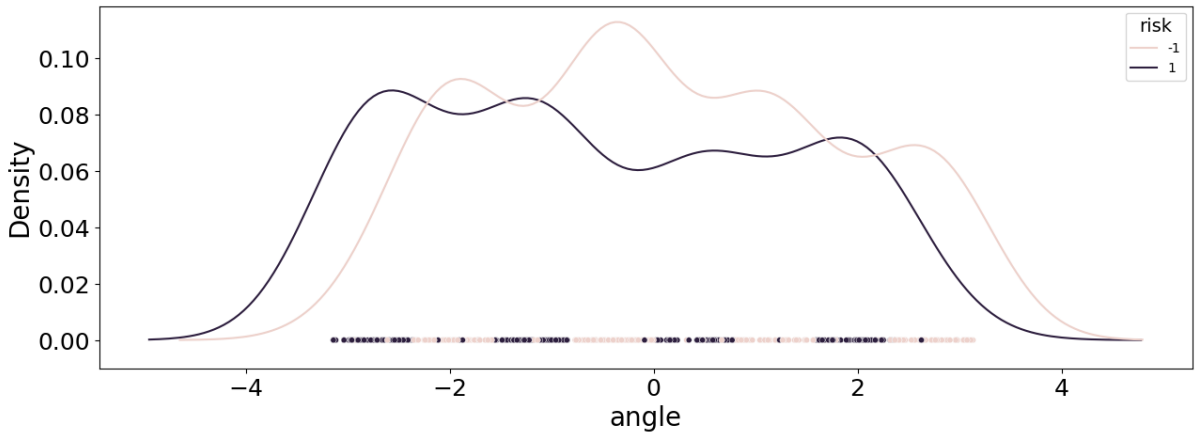
We can see that our datapoints have a windmill like pattern, and we'd probably get a good accuracy with the right model, perhaps SVM with a rbf kernel.

Additionally we can look at our dataset from a radian perspective and make our decision based on the angle of each point, as we can see that our decision regions should be in the format of $\theta_1 \leq \theta < \theta_2$, with this perspective in mind we'll get 8 alternating decision regions for almost every $45°$.



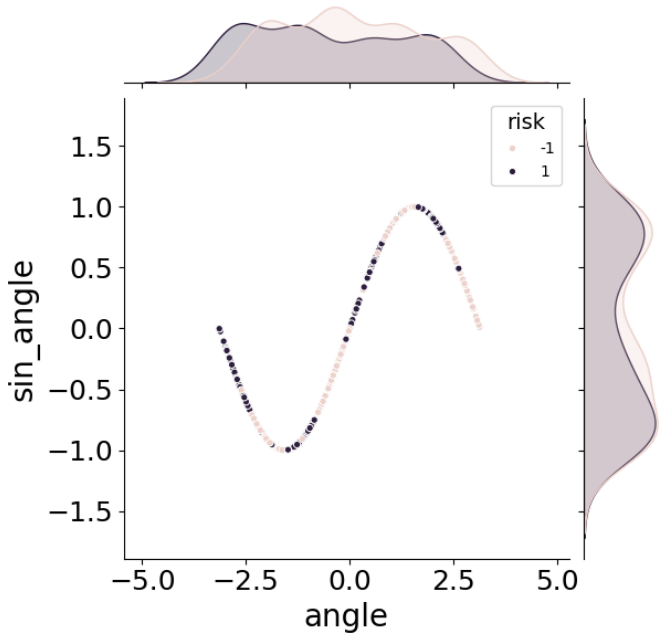
Q17

We can see in this plot the Density of our datapoints label's by angle [rad], here its more apparent that for specific angles we have a higher density of a certain label, as we said before we got 8 separate regions where we can give each one a certain label and get good accuracy, but still we can't linearly separate the datapoints



Q18

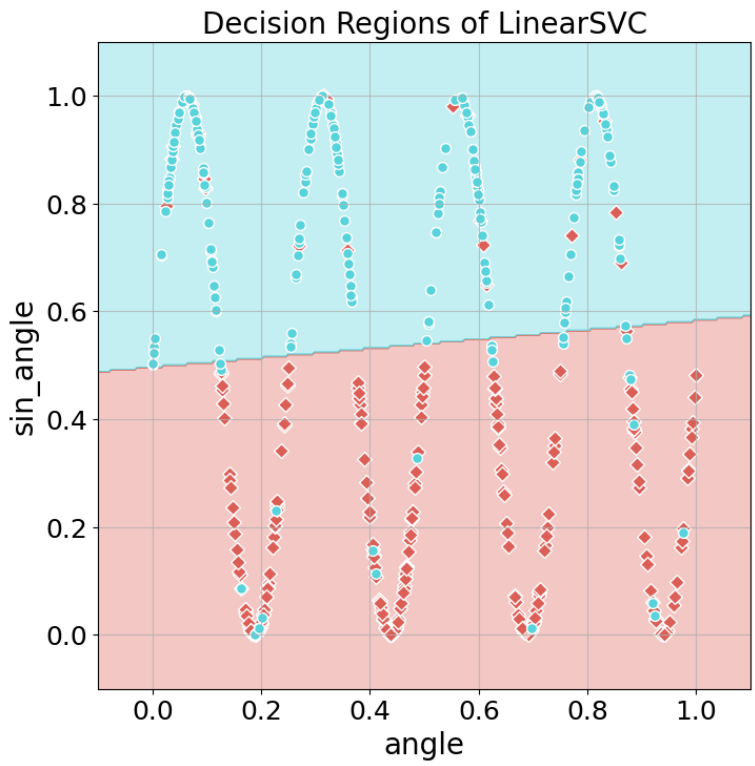
The data still doesn't seem linearly separable but playing with the beta hyperparameter might get us there, as that could modify the distribution of the data points in the plot.



Q19

The beta controls the frequency of our sin function, looking at the plot from the previous question we can see that we have 4 regions of each label and they are pretty much equally divided along the sin function, By selecting an appropriate beta value, we aim to achieve a transformation that aligns all data points of the same label above a certain separation threshold.
Given the four regions per label, a beta value of 4 sounds reasonable. This choice is motivated by its potential to reposition the data such that all datapoints belonging to the blue label lie above the y=0.5 separation line. Consequently, this adjustment would facilitate a more straightforward linear separation between the two classes.
We tried multiple values around beta = 4 and found out that beta fits the problem beautifully.

The accuracy we got after the radian transformation is way better than the one we had before as can be seen from the results below compared to the 0.53 accuracy from before.

Training Accuracy: 0.906

Test Accuracy: 0.911